



Instituto Tecnológico de Costa Rica
Centro Académico de Alajuela
Bases de datos 2

Proyecto 2 Bases de datos 2

Josue Mena – Carné: 2022138381
Antonio Fernández García – Carné: 2022075006
Maximilian Latysh – Carné: 2022091544

Profesor:
Alberto Shum Chan

II Semestre, 2023

Introducción

La inteligencia de negocios combina análisis de negocios, minería de datos, visualización de datos, herramientas e infraestructura de datos, y las prácticas recomendadas para ayudar a las organizaciones a tomar decisiones basadas en los datos. El presente proyecto tiene como objetivo trabajar de forma básica las diferentes partes de la inteligencia de negocio y las tecnologías necesarias para lograrlo, además de replicar y montar una base Postgresql.

Para lograr lo anterior se utilizará la [base de datos de ejemplo](#) “dvdrental”, sobre esta se realizará el proceso de replicación escogido por el grupo, de modo que se tenga una base maestra y otra base esclava que sirva como backup.

Adicionalmente, se diseñará un modelo multidimensional que cumpla con las características necesarias para funcionar como un datamart. Esto permitirá organizar la información de manera eficiente, facilitando un acceso rápido y estructurado a los datos relevantes para la toma de decisiones estratégicas.

Por último, la visualización de datos se llevará a cabo mediante el uso de dashboards en el software Tableau, dichos dashboards permiten que el usuario pueda ver fácilmente los datos que necesita para analizar y de esta forma tomar decisiones para el negocio.

Descripción del proyecto

Para dar inicio a la implementación del proyecto se mantuvo el orden que se encontró en la especificación. El primer punto de suma importancia fue descargar la base de datos y restaurarla al motor de PostgreSQL de los 3 integrantes. Luego se procedió a crear las funciones/procedimientos almacenados para: insertar un nuevo cliente, registrar un alquiler, registrar una devolución y buscar una película por ID. Todos los mencionados son bastante sencillos y claros y llevan un proceso bastante simple; en el caso de registrar una devolución va un poco más allá, ya que es necesario calcular cuántos días de alquiler fueron.

Posterior a lo mencionado, se pasó a crear tanto los roles como usuarios. En este caso fueron los siguientes roles: EMP y ADMIN y los usuarios: video, empleado1 y administrador1. Ya con todo esto configurado, es posible pasar a la replicación de la base de datos. La técnica de replicación usada fue replicación lógica por medio de publicaciones y suscripciones en la base de datos. El procedimiento de esta parte se puede ver más a fondo en su respectiva sección.

La replicación lógica consiste en que desde un servidor principal se puede hacer una publicación y mandar datos en dicha publicación que sean de interés para replicar con sus respectivos permisos. Luego, desde un servidor diferente que es secundario se puede crear una base de datos con las mismas tablas pero estas están vacías. Con eso, se crea una suscripción desde esta base de datos que llame a la publicación creada anteriormente y se van a cargar los datos correspondientes a cada tabla.

El siguiente paso fue el desarrollo del modelo multidimensional. Esta parte se puede ver más a fondo en la sección que le toca. En general, se trabajó con las dimensiones: película, lugar, fecha y sucursal y se trabajó con el hecho de alquileres. Luego, con esas tablas creadas se procedió a llenarlas por medio de procedimientos almacenados.

Con el punto anterior, se procedió a la realización del dashboard por medio del software Tableau. Con cada hecho y dimensión se pudieron transformar los datos para dar una visualización bastante clara y efectiva sobre lo que se estaba trabajando. Todo esto, claramente adaptado a lo solicitado.

Realizado todo lo anterior se llegó a la finalización del proyecto de forma satisfactoria, con cada punto desarrollado profundamente. En el resto del documento actual se puede ver más a fondo puntos de vital importancia explicados más a fondo.

Documentación del proceso de replicación

Antes de empezar con la descripción del proceso de replicación, hace falta especificar que el sistema operativo utilizado fue macOS Sonoma. Por lo tanto, las partes de crear, configurar (internamente), iniciar y parar las instancias de los dos servidores (maestra y esclava) son parcialmente específicas para este sistema operativo. Sin embargo, las partes hechas en la aplicación pgAdmin 4 (entre las cuales se encuentran conectar con los dos servidores, “restaurar” las bases de datos, “trunquear” el esclavo, ejecutar el código sql correspondiente y crear el sistema de replicación) se pueden generalizar.

Para iniciar todo, se decidió utilizar la replicación lógica para poder replicar solamente una base de datos de un servidor y no todas (en contraste con replicación de streaming). En primera instancia, se deben tener dos servidores en la misma máquina. Se les llamará servidor-1 (principal) y servidor-2 (réplica) a lo largo de este documento. Para hacer esta tarea se ejecutaron los siguientes comandos en el directorio donde se estaba realizando el proyecto, con la finalidad que cada servidor esté en un directorio único:

```
% mkdir -p servidor-1/data
% mkdir -p servidor-2/data
```

Posteriormente, se puede proceder a la creación de las plantillas de ambos servidores con el comando initdb:

```
% initdb -D servidor-1/data -U postgres -W -A trust -E UTF8 -X
$PWD/servidor-1/data
% initdb -D servidor-2/data -U postgres -W -A trust -E UTF8 -X
$PWD/servidor-2/data
```

Luego, por medio de un script de python se decidió buscar algún puerto libre en la máquina, esto con el fin de configurar el servidor de replicación ahí:

```
% python3
```

```
>>> import socket
>>> s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
>>> s.bind(('', 0))
>>> addr = s.getsockname()
>>> print(addr[1])
>>> s.close()
>>> quit()
```

Teniendo ya un puerto disponible se procede a modificar los archivos de configuración en ambos servidores llamados “postgresql.conf”, donde había que ajustar el “wal_level” a “logical”, quitar el comentario en la línea del elemento “port” y para servidor-2 actualizar el puerto en donde va a estar funcionando.

Luego, se procede a inicializar ambos servidores con los comandos:

```
% pg_ctl -D $PWD/servidor-1/data start
% pg_ctl -D $PWD/servidor-2/data start
```

A partir de este punto se procederá a utilizar la herramienta “PGAdmin4” para realizar la replicación lógica, que trabaja por medio de publicaciones y suscripciones. De forma nativa, con comandos de consola esto se puede realizar, pero a nivel grupal se tomó la decisión de implementarlo por medio de la interfaz gráfica, ya que la implementación se puede observar de forma más clara. Entrando a la herramienta se registran ambos servidores:

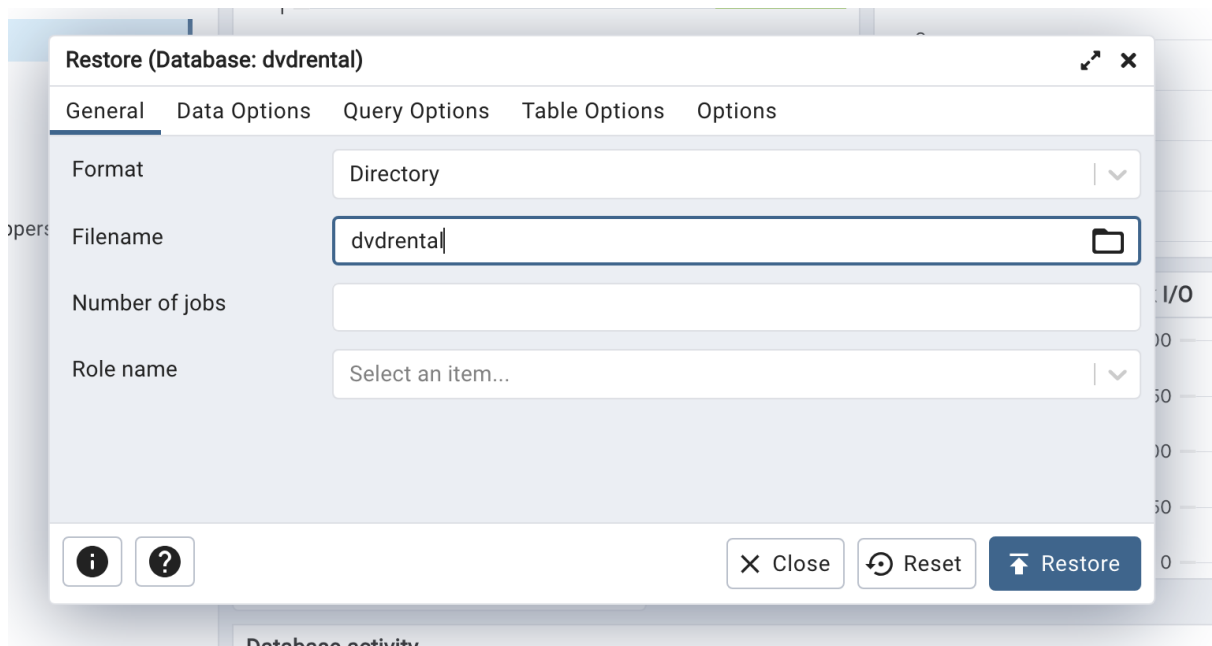
The screenshot shows the 'Register - Server' dialog box in PGAdmin4, with the 'Connection' tab selected. The dialog has a title bar with 'Welcome' and a close button. The tabs are 'General', 'Connection', 'Parameters', 'SSH Tunnel', and 'Advanced'. The 'Connection' tab contains the following fields and controls:

- Host name/address: localhost
- Port: 5432
- Maintenance database: postgres
- Username: postgres
- Kerberos authentication?: ☐
- Password: (password field)
- Save password?: ☒
- Role: (empty text field)
- Service: (empty text field)

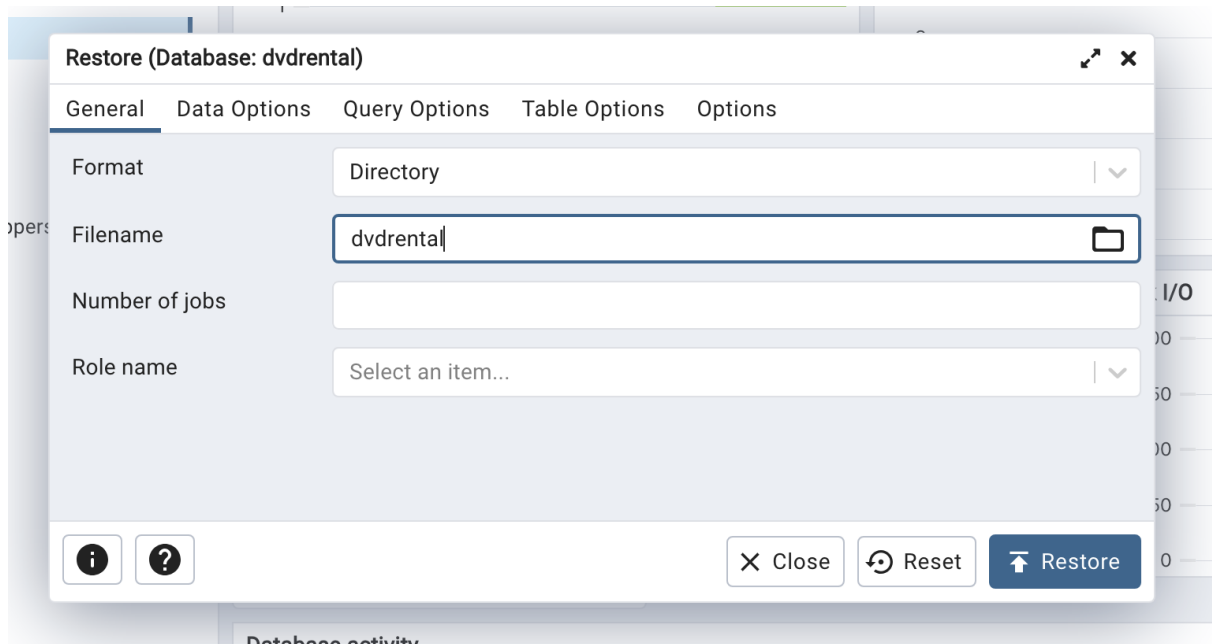
At the bottom of the dialog, there are three buttons: 'Close', 'Reset', and 'Save'.

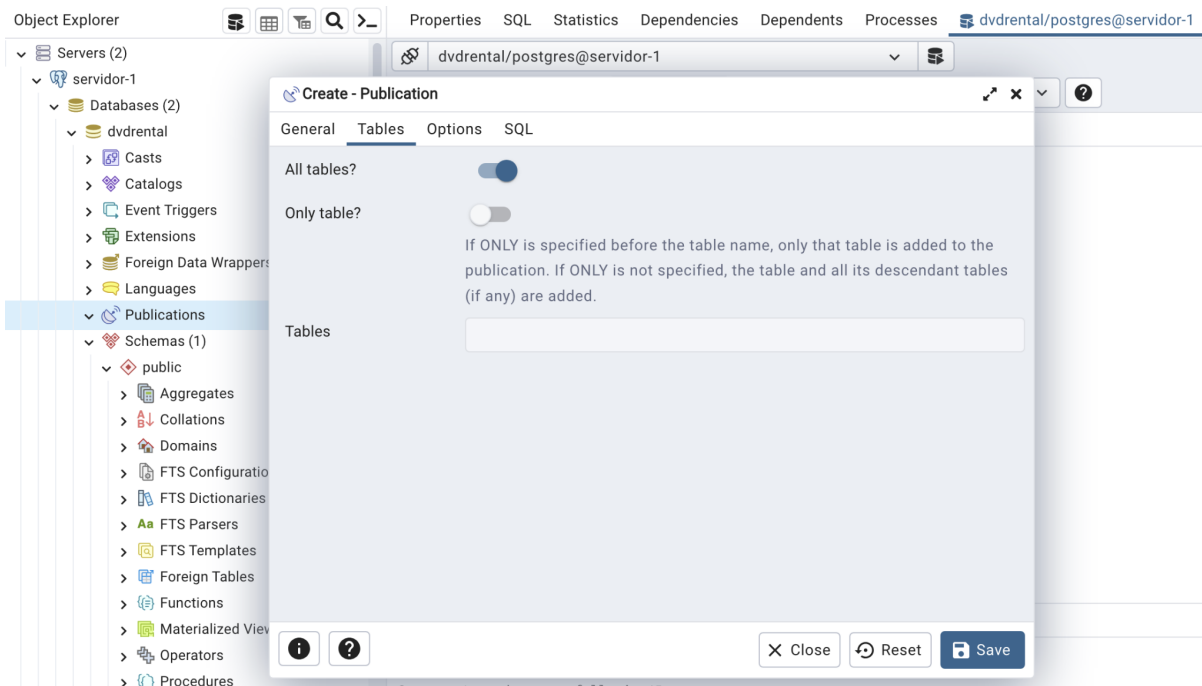
- ✓ Servers (2)
 - > servidor-1
 - > servidor-2

Teniendo ambos servidores se procede restaurar la base de datos “dvdrental” en el servidor-1:

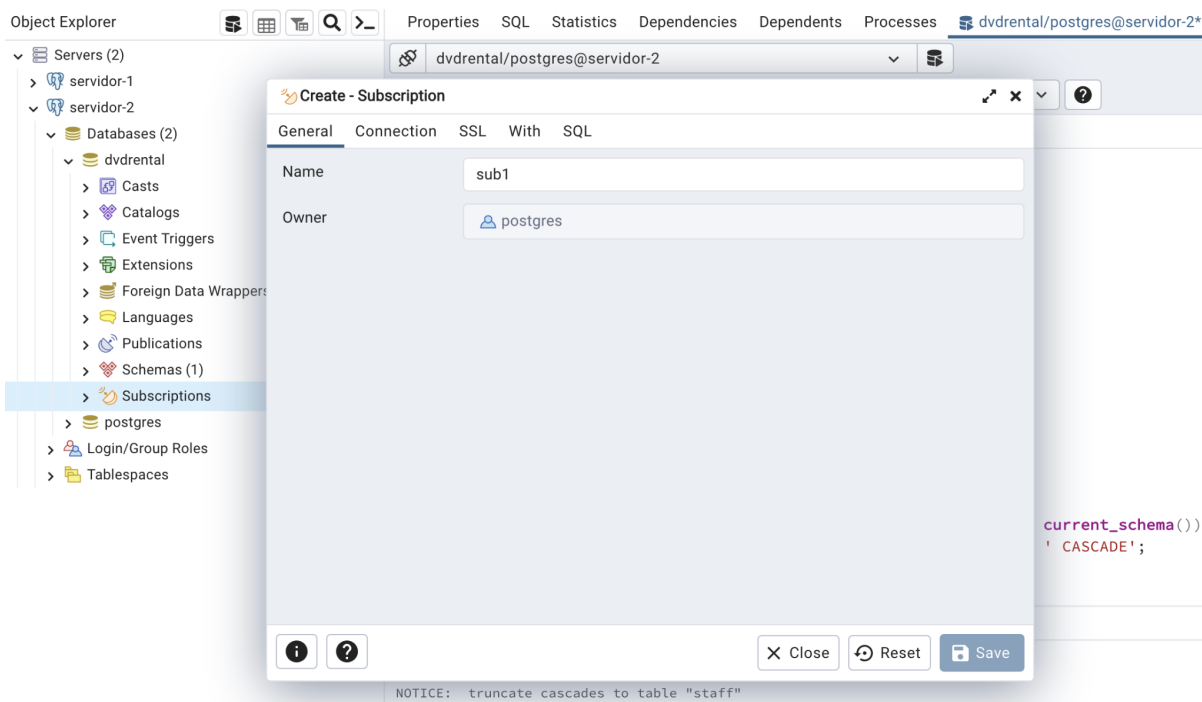


En este punto se ejecuta el archivo “postgre.sql”. Aquí se crean los procedimientos almacenados, roles, usuarios, tablas de dimensión y hechos. Se crea la publicación de la base de datos por medio de la herramienta mencionada anteriormente:

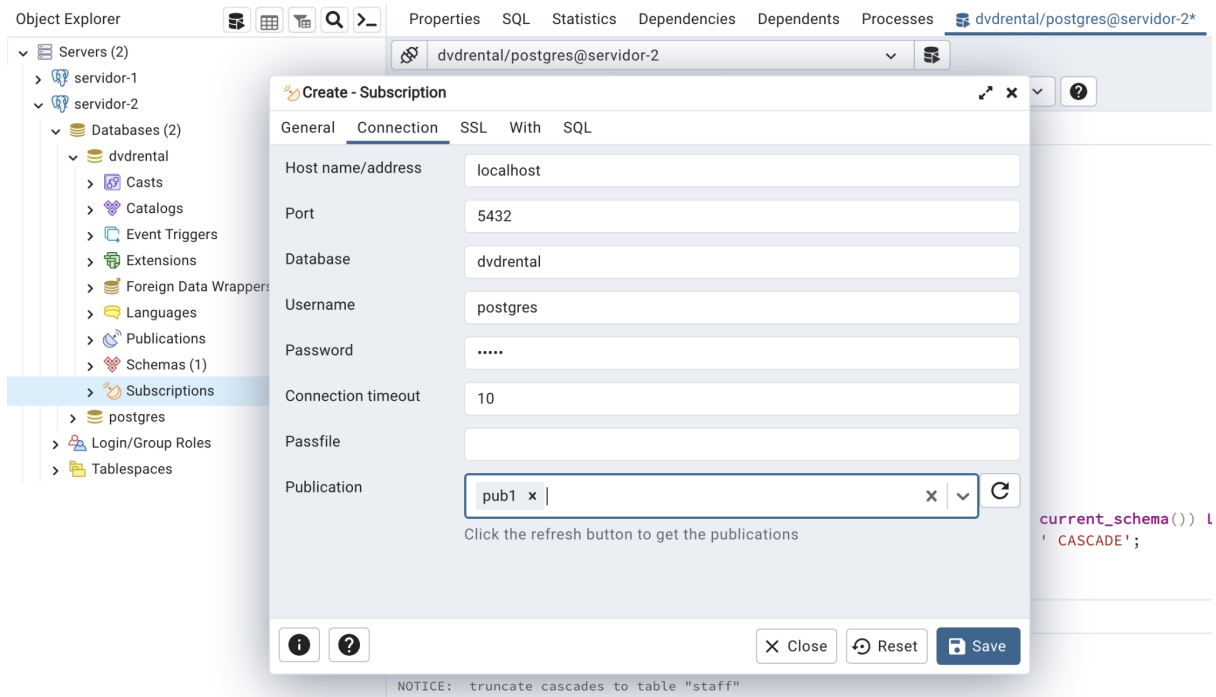




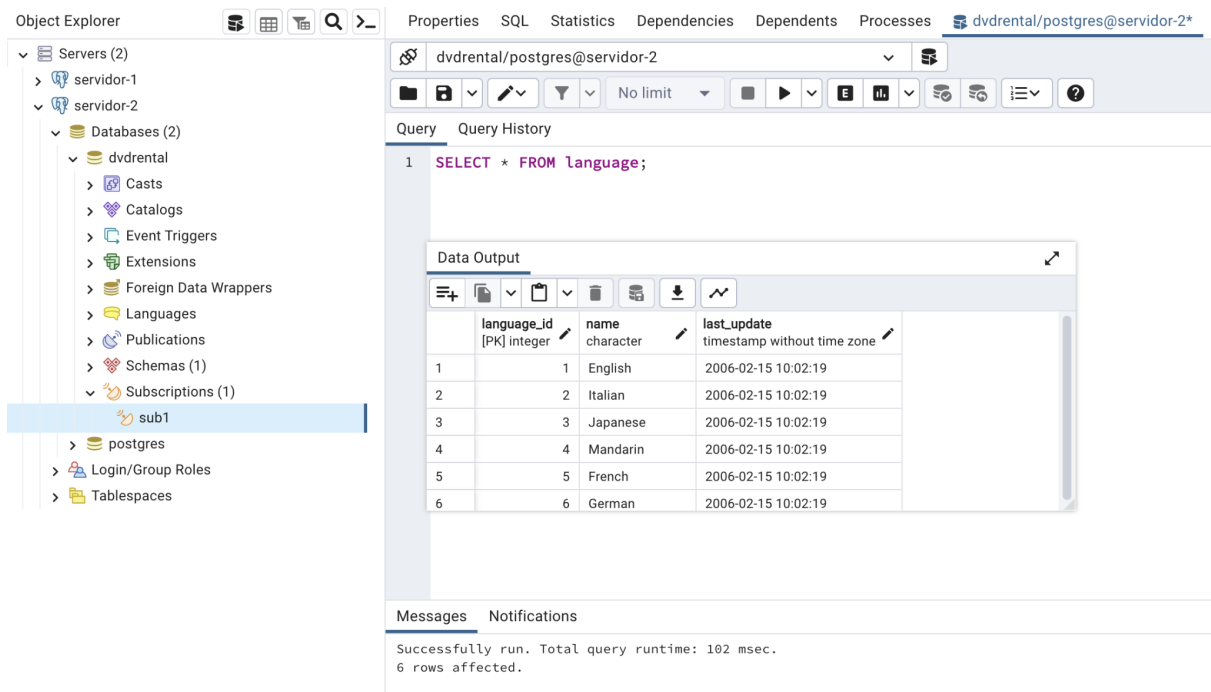
Se prosigue creando la suscripción en el servidor de replicación, pero antes de esto se debe crear la base de datos y crear las tablas, de forma que estén vacías, en el servidor de replicación. Una vez realizado este proceso, se crea una suscripción en dicha base de datos del servidor-2:



Se accede a la publicación creada anteriormente en la base de datos “dvdrental”:



Luego, se verifica que los datos fueron replicados exitosamente, haciendo una consulta a la tabla de idiomas:



Ante esto, las tablas de hechos y dimensiones se encuentran vacías, por lo que se procede a alimentarlas desde servidor-2:

The screenshot shows the pgAdmin interface with the 'Messages' pane open. The query history displays a series of five 'CALL' statements executed on 'servidor-2':

```

1 CALL poblar_dim_film();
2 CALL poblar_dim_address();
3 CALL poblar_dim_rental();
4 CALL poblar_dim_store();
5 CALL poblar_hechos_alquileres();

```

The 'Messages' pane shows the following output:

```

CALL
Query returned successfully in 148 msec.

```

The 'Object Explorer' on the left shows the 'Tables (15)' folder expanded, listing tables like actor, address, category, city, country, and customer.

Al hacer esto se comprueba nuevamente que los datos se actualicen de forma correcta en servidor-2:

The screenshot shows the pgAdmin interface with the 'Data Output' pane open. The query history displays a 'SELECT' statement executed on 'servidor-2':

```

1 CALL poblar_dim_film();
2 CALL poblar_dim_address();
3 CALL poblar_dim_rental();
4 CALL poblar_dim_store();
5 CALL poblar_hechos_alquileres();
6
7 SELECT * FROM hechos_alquileres;

```

The 'Data Output' pane shows the following table:

	film_id integer	address_id integer	rental_id integer	store_id integer	total int
1	901	262	13218	1	
2	476	100	14462	2	
3	647	164	10305	1	
4	625	50	13226	1	
5	438	301	9249	2	
6	479	266	2392	1	
7	821	385	3099	2	
8	722	259	12206	1	
9	260	132	2519	2	
10	54	195	1677	2	
11	48	216	2739	1	
12	841	24	14613	2	
13	253	93	1252	2	
14	663	133	2768	1	

The 'Messages' pane shows the following output:

```

Successful
14593 rows

```

The 'Object Explorer' on the left shows the 'Tables (15)' folder expanded, listing tables like actor, address, category, city, country, and customer.

Cabe mencionar que en servidor-2 se pueden hacer consultas en todas las tablas e inserciones en las tablas de dimensiones y hechos únicamente.

Documentación del modelo multidimensional

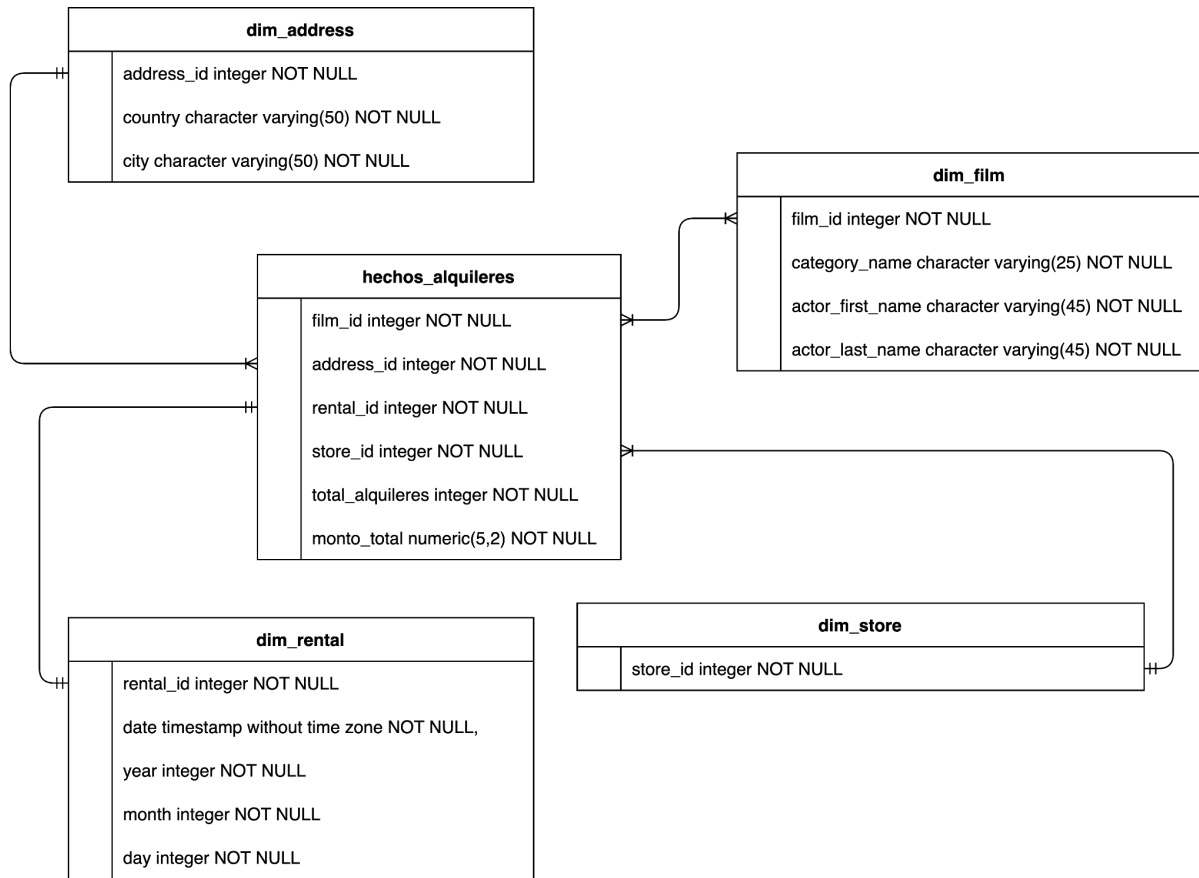


Diagrama en forma del esquema estrella.

Documentación de las dimensiones y sus atributos

1. Address (lugar)

La ubicación del cliente que hizo el alquiler.

Atributo	Definición	Valores de ejemplo
address_id	La identificación del lugar	1123, 42, 024, 56
country	El país del lugar	Russia, Costa Rica, Kazakhstan
city	La ciudad de la ubicación	Moscow, Grecia, Astana

2. Film (película)

Un join con las películas, sus categorías y sus actores.

Atributo	Definición	Valores de ejemplo
film_id	La identificación de la película	1123, 42, 024, 56
category_name	El nombre de la categoría	Action, Comedy
actor_name	El nombre completo del actor	Grigoriy Leps, Vladimir Vysotski, Yuriy Nikolin
actor_first_name	El nombre del actor	Grigoriy, Vladimir, Yuriy
actor_last_name	El apellido del actor	Leps, Vysotski, Nikolin

3. Rental (alquiler)

El alquiler que contiene la fecha en específico.

Atributo	Definición	Valores de ejemplo
rental_id	La identificación del alquiler	1123, 42, 024, 56
date	La fecha del alquiler	'2023-10-16 14:30:00', '2023-10-18 14:30:00'
year	El año del alquiler	2023, 2024
month	El mes del alquiler	1, 12
day	El día del alquiler	5, 27

4. Store (sucursal)

La sucursal de donde se hizo la compra.

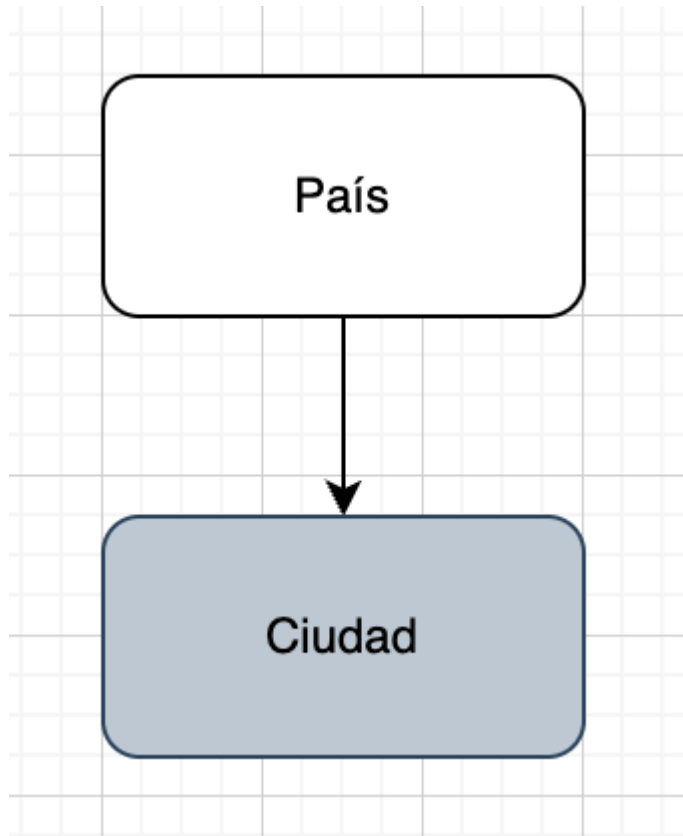
Atributo	Definición	Valores de ejemplo
store_id	La identificación de la sucursal	1123, 42, 024, 56

Análisis de medidas

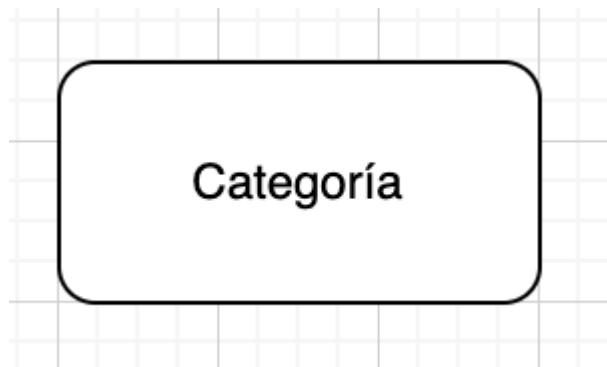
Nombre de la medida	Descripción	Tipo Medida	Regla de agregación	Formula
Cantidad total de alquileres	El número total de alquileres	Hecho base	Cuenta	count(rental_id)
Monto total de alquileres	El monto total de un alquiler	Hecho base	Suma	sum(payment.amount)

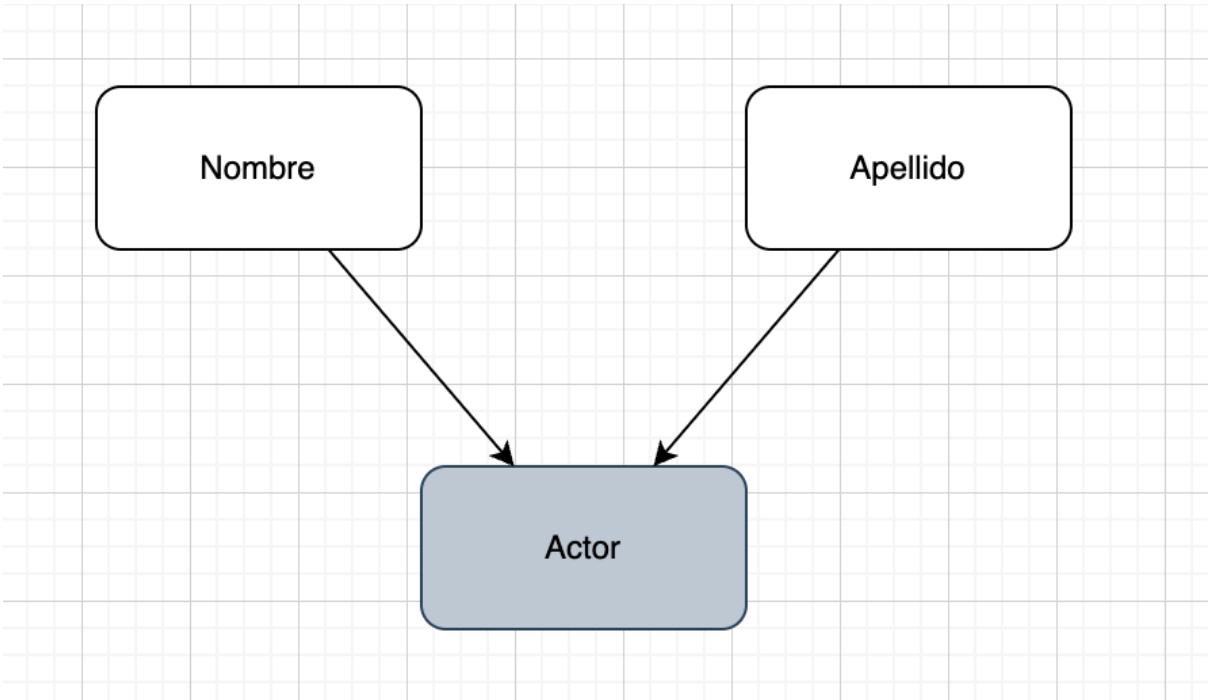
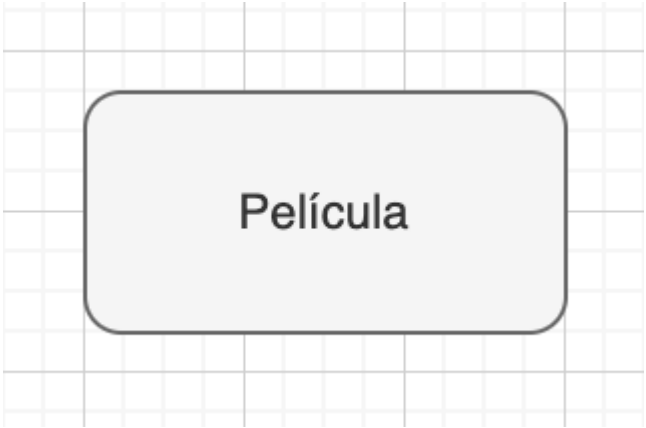
Análisis de la granularidad

1. Address (lugar)

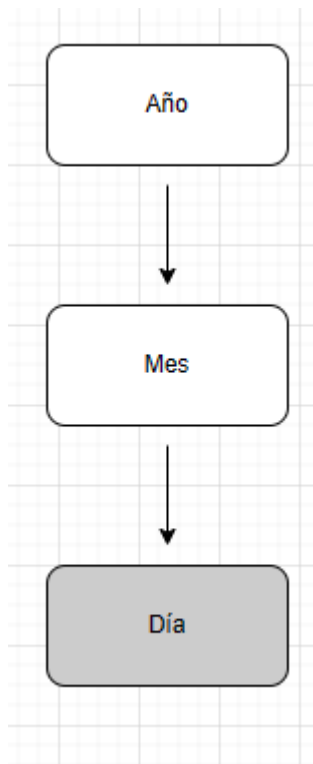


2. Film (película)

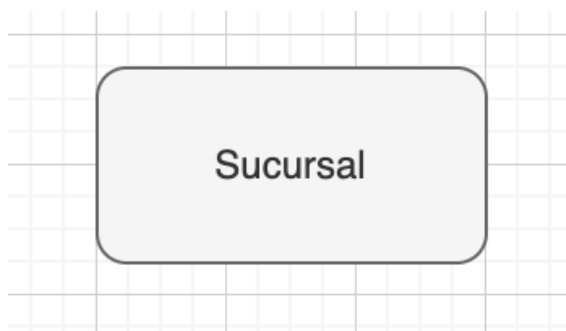




3. Rental (alquiler)



4. Store (sucursal)



Fuente de datos

1. Address (lugar)

Atributo	Tabla de origen	Atributo de origen
address_id	address	address_id
country	country	country
city	city	city

2. Film (película)

Atributo	Tabla de origen	Atributo de origen
film_id	film	film_id
category_name	category	category_name
actor_name	actor	actor_first_name && actor_last_name
actor_first_name	actor	actor_first_name
actor_last_name	actor	actor_last_name

3. Rental (alquiler)

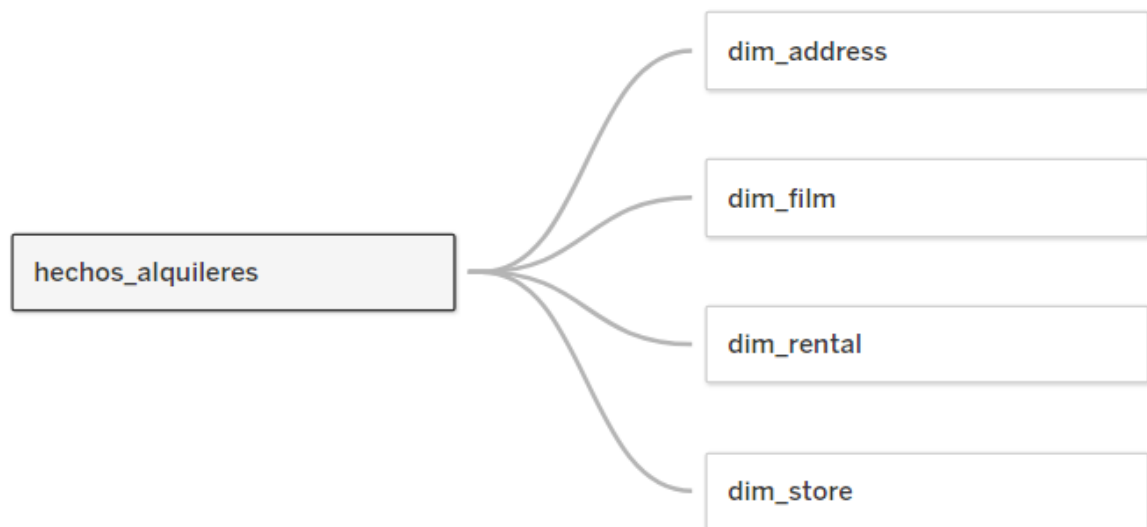
Atributo	Tabla de origen	Atributo de origen
rental_id	rental	rental_id
date	rental	rental_date
year	rental	rental_date
month	rental	rental_date
day	rental	rental_date

4. Store (sucursal)

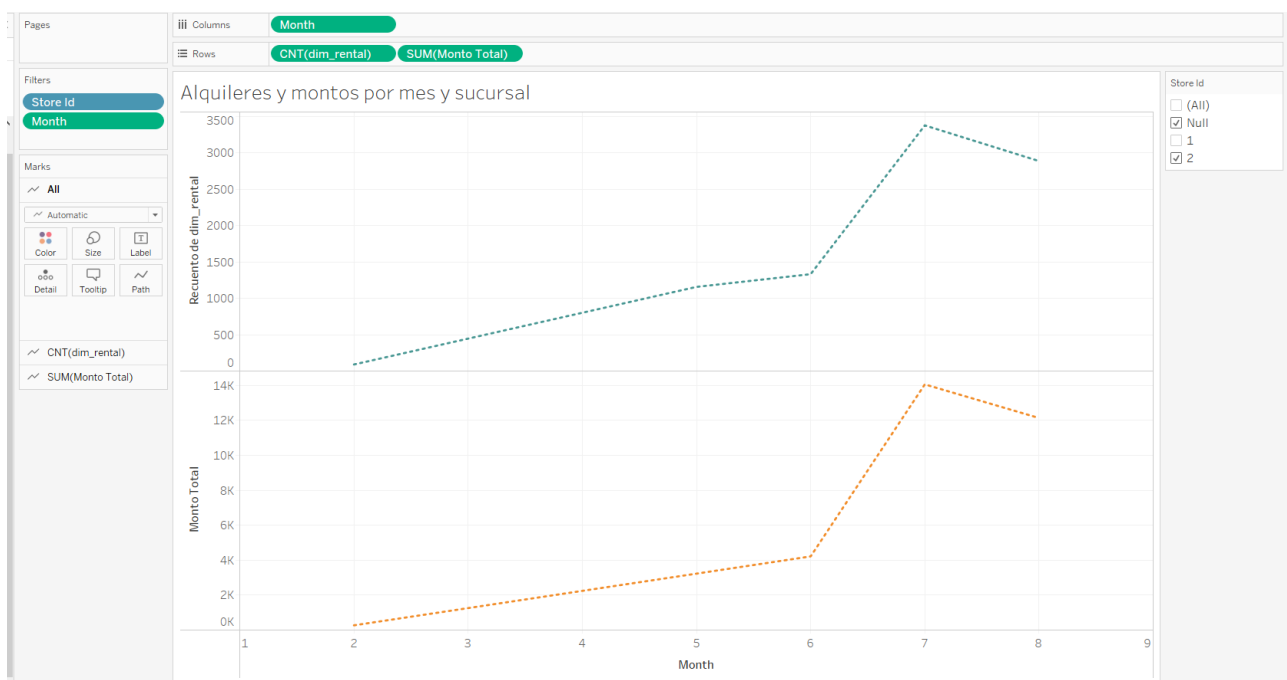
Atributo	Tabla de origen	Atributo de origen
store_id	store	store_id

Documentación del proceso de visualización de datos mediante Tableau

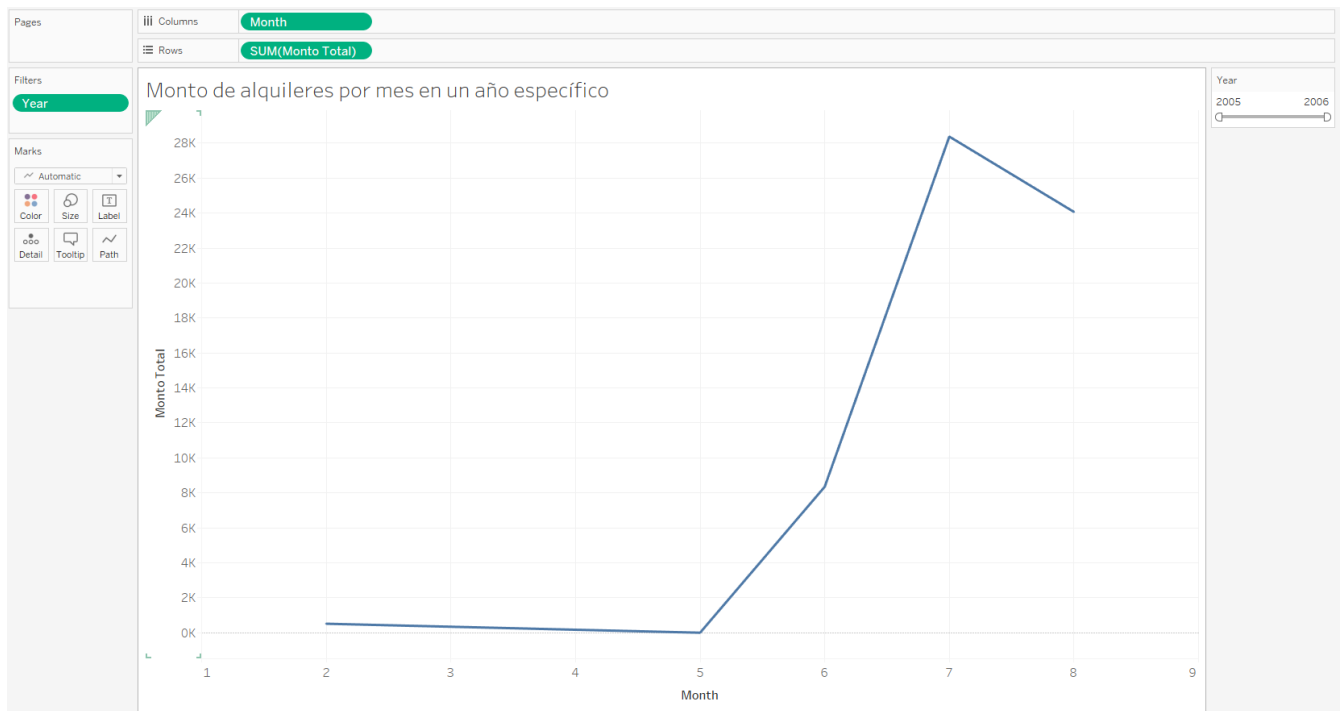
Lo primero que se debe hacer es arrastrar las dimensiones y el hecho a la parte del Datasource, como se puede ver en la siguiente imagen, las dimensiones se relacionan de manera automática con el hecho.



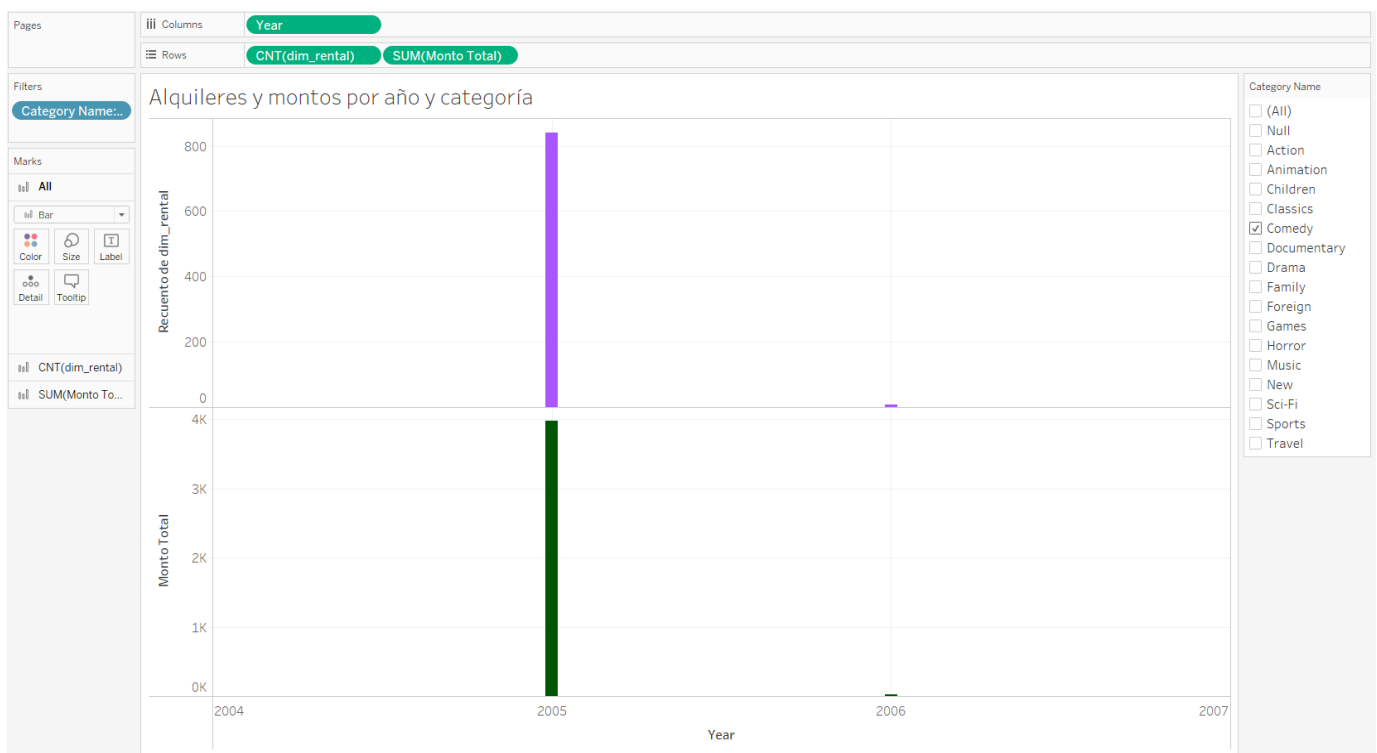
Después se deben crear las diferentes sheets donde se hará la graficación de los datos según corresponda, en el primer caso se utilizará el store_id como un filtro, gracias a esto se tiene la opción de elegir una sucursal o ambas, en las columnas 'month' permite separar por mes la cantidad de alquileres y el monto recaudado, estos dos últimos van en la parte de las filas.



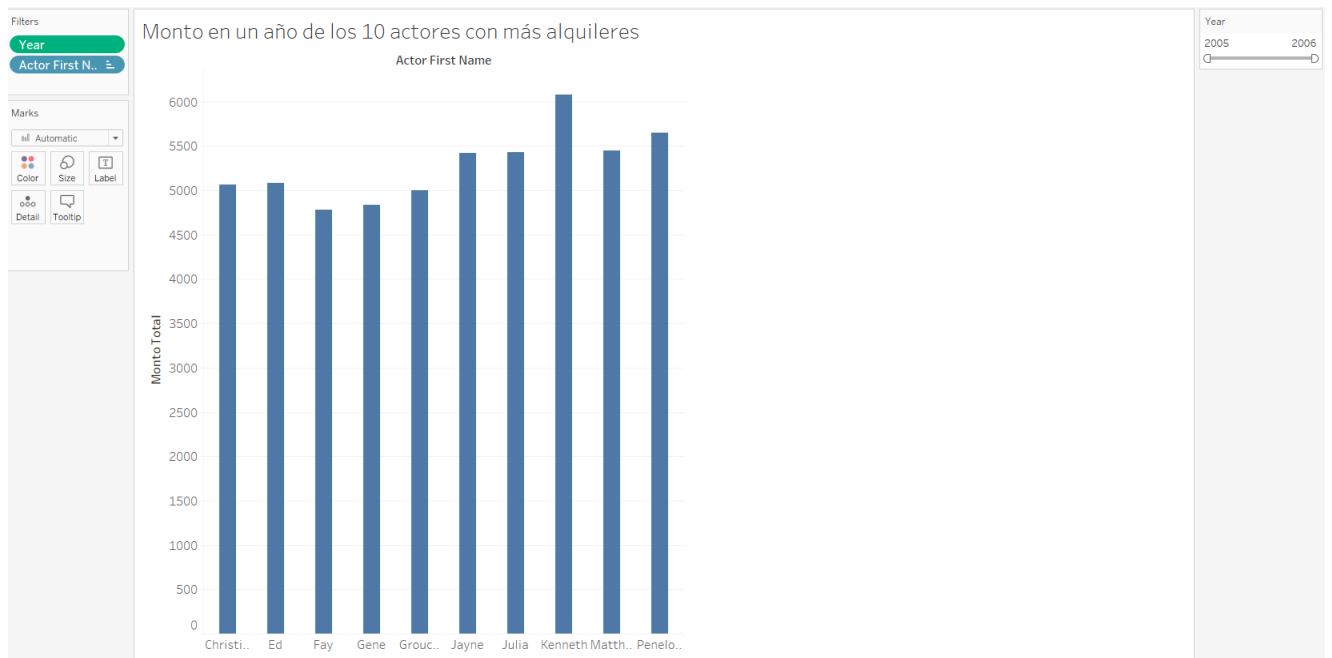
El siguiente gráfico representa el monto de alquileres por mes en un año específico, para esto se utiliza el campo Year como un filtro, esto brinda un rango en años que permite seleccionar el que desea, al igual que el anterior, 'month' permite separar por mes el monto recaudado, este último va en la parte de las filas.



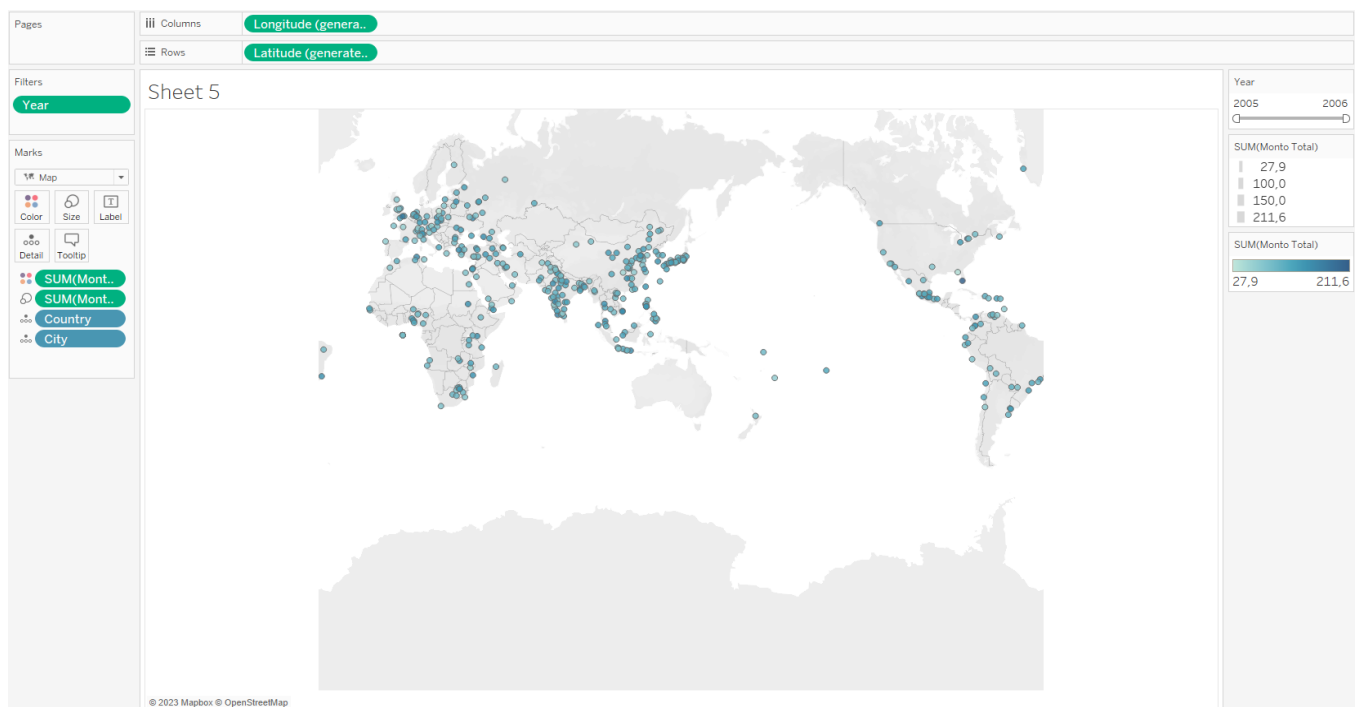
En el tercer gráfico se necesita mostrar la cantidad de alquileres y el monto recaudado según el año y una categoría de película, para esto se usa la categoría de la película como un filtro, lo que permite elegir la categoría que se desee, en las columnas se usa Year para separar en los distintos años mientras que en las filas se coloca la cantidad de alquileres y el monto recaudado.



El el cuarto gráfico se necesita el monto en un año específico de los 10 actores con más alquileres, para esto se usa Year como filtro para tener un rango de años y elegir el que se desee, en las columnas se utiliza el campo Actor First Name para separar los actores, por último en las filas se utiliza la cantidad de alquileres pero ordenada, de modo que solo muestre los 10 más altos.



Por último se desea observar en un mapa las ciudades y el monto generado en un año o varios, de modo que cada punto representa una ciudad y el tamaño del punto está relacionado con la cantidad, es decir, entre más grande es porque en esa ciudad hubo un mayor monto. Para lograr esto se usa Year como filtro, así permite elegir el rango de años o el año, después se utilizan los campos autogenerados Longitude, Latitude para dibujar el mapa, en la parte de “Details” se usan los campos Country y City, de esta manera Tableau sabe dónde debe colocar los puntos, además se usa el campo de monto total para definir el tamaño y color de los puntos, logrando un resultado claro de analizar.



Conclusión

Una vez realizadas todas las tareas del proyecto entre los 3 integrantes se llegó a varias conclusiones con respecto a la realización del mismo.

1. La herramienta Tableau, aunque tiende a ser relativamente lenta en términos de la rapidez con la que ejecuta las consultas a la base de datos, es bastante útil y poderosa a la hora de trabajar con inteligencia de negocios.
2. A la hora de hacer la replicación y sus fases preparatorias, se descubrió que el procedimiento resulta ser más cómodo cuando se hace por medio del CLI y el GUI. Utilizando el CLI, se logró configurar los servidores de una forma muy “fácil”. Después, el GUI permitió ejecutar la pre-restauración de los servidores junto con la replicación requerida. Por lo tanto, para futuros proyectos sería una buena idea considerar los dos tipos de interfaces con el fin de promover la finalización más efectiva.
3. PostgreSQL terminó siendo más cómodo a la hora de programar y conectar con otros sistemas que algunos de los RDBMS previamente analizados en el curso.
4. Los modelos multidimensionales son relativamente más fáciles de implementar de lo que se esperaba. Por lo tanto, para futuros casos de BI, van a ser considerados con más interés.

El SQL

```
--
=====
=
-- Punto 1: Hacer procedimientos, funciones, roles y usuarios.

--
-----
-
-- Procedimientos y funciones

-- Procedimiento para registrar un alquiler

CREATE OR REPLACE PROCEDURE insertar_cliente(
    p_store_id smallint,
    p_first_name character varying,
    p_last_name character varying,
    p_email character varying,
    p_address_id smallint,
    p_activebool boolean,
    p_create_date date,
    p_last_update timestamp without time zone,
    p_active integer)
SECURITY DEFINER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO customer (
        store_id,
        first_name,
        last_name,
        email,
        address_id,
        activebool,
        create_date,
        last_update,
        active
    ) VALUES (
        p_store_id,
        p_first_name,
        p_last_name,
        p_email,
        p_address_id,
```

```

        p_activebool,
        p_create_date,
        p_last_update,
        p_active
    );
END;
$$;

-- Procedimiento para registrar un alquiler

CREATE OR REPLACE PROCEDURE insertar_renta(
    p_rental_date timestamp without time zone,
    p_inventory_id integer,
    p_customer_id smallint,
    p_return_date timestamp without time zone,
    p_staff_id smallint,
    p_last_update timestamp without time zone
)
SECURITY DEFINER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO rental (
        rental_date,
        inventory_id,
        customer_id,
        return_date,
        staff_id,
        last_update
    ) VALUES (
        p_rental_date,
        p_inventory_id,
        p_customer_id,
        p_return_date,
        p_staff_id,
        p_last_update
    );
END;
$$;

-- Procedimiento para registrar una devolución

CREATE OR REPLACE FUNCTION buscar_pelicula(p_film_id integer)
RETURNS SETOF film AS
$$
DECLARE

```

```

        resultado film; -- Declarar una variable para almacenar el
resultado
BEGIN
    -- Realizar la consulta y almacenar el resultado en la variable
'result'
    SELECT * INTO resultado FROM film WHERE film_id = p_film_id;

    -- Devolver el resultado
    RETURN NEXT resultado;
END;
$$
LANGUAGE plpgsql;

-- Procedimiento para registrar una devolución

CREATE OR REPLACE PROCEDURE registrar_devolucion(
    p_customer_id smallint, -- id del cliente
    p_staff_id smallint, --id del staff
    p_rental_id integer, -- id de la renta
    p_payment_date timestamp without time zone -- es now
)
SECURITY DEFINER
LANGUAGE plpgsql
AS $$
DECLARE
    dias integer; -- dias totales
    fecha_renta timestamp without time zone; -- la fecha almacenada en
la tabla rental
    p_amount numeric(5,2); -- cantidad a pagar
    p_rate numeric(4,2); -- el costo por dia
BEGIN

    select rental_date into fecha_renta from rental where rental_id =
p_rental_id; -- asignamos la fecha renta

    dias := DATE_PART('day', p_payment_date - fecha_renta); --
calculamos los dias en integer

    select rental_rate into p_rate from film where film_id = (select
film_id from inventory where inventory_id = (select inventory_id from
rental where rental_id = p_rental_id)); -- asignamos el costo por dia
    p_amount := LEAST(dias * p_rate, 999.99); -- la cantidad a pagar
se calcula pagando 0.99 por dia

-- se insertan los datos
    INSERT INTO payment (customer_id, staff_id, rental_id, amount,

```

```

payment_date) VALUES (
    p_customer_id,
    p_staff_id,
    p_rental_id,
    p_amount,
    p_payment_date
);

END;
$$;

--
-----
-
-- Roles

-- El rol de empleado
CREATE ROLE EMP;

GRANT EXECUTE ON PROCEDURE insertar_renta TO EMP;
GRANT EXECUTE ON PROCEDURE registrar_devolucion TO EMP;
GRANT EXECUTE ON FUNCTION buscar_pelicula(INTEGER) TO EMP;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM EMP;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM EMP;

-- El rol de administrador
CREATE ROLE ADMIN;

GRANT EXECUTE ON PROCEDURE insertar_cliente TO ADMIN;
GRANT EMP TO ADMIN;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM ADMIN;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM ADMIN;

--
-----
-
-- Usuarios

-- El usuario video
CREATE USER video NOLOGIN;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO video;
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO video;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO video;

-- El empleado 1
CREATE USER empleado1;

```

```

GRANT EMP TO empleado1;

-- El administrador 1
CREATE USER administrador1;
GRANT ADMIN TO administrador1;

--
-----
-
-- Alterar procedimientos y funciones

ALTER PROCEDURE insertar_cliente OWNER TO video;
ALTER PROCEDURE registrar_devolucion OWNER TO video;
ALTER FUNCTION buscar_pelicula(integer) OWNER TO video;
ALTER PROCEDURE insertar_renta OWNER TO video;

--
-----
-
-- Pruebas

call insertar_cliente(
    1 ::smallint,
    'Pedro'::character varying,
    'Ramirez' ::character varying,
    'pedro@example.com' ::character varying,
    1 ::smallint,
    true,
    '2023-10-16'::date,
    now() ::timestamp without time zone,
    1
);
call insertar_renta(
    '2023-10-16 14:30:00'::timestamp without time zone,
    1, -- Reemplaza con el ID de inventario deseado
    1 ::smallint, -- Reemplaza con el ID de cliente deseado
    '2023-10-18 14:30:00'::timestamp without time zone, -- Fecha de
retorno
    1 ::smallint, -- Reemplaza con el ID de personal deseado
    '2023-10-16 14:30:00'::timestamp without time zone
);
select * from buscar_pelicula(4);
call registrar_devolucion(1 ::smallint, 1 ::smallint, 1, now()
::timestamp without time zone);

--

```



```

-----
-
-- Eliminar procedimientos, funciones, usuarios y roles

DROP PROCEDURE insertar_cliente;
DROP PROCEDURE insertar_renta;
DROP PROCEDURE registrar_devolucion;

DROP FUNCTION buscar_pelicula(integer);

DROP USER empleado1;
DROP USER administrador1;

REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM video;
REVOKE ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public FROM video;
REVOKE ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public FROM video;
DROP USER video;

DROP ROLE EMP;
DROP ROLE ADMIN;

--
=====
=
-- Punto 2: Hacer la replicación de la base de datos.
-- Punto 3: Modelo multidimensional sobre la base de datos de ejemplo de
PostgreSQL.

--
-----
-
-- Las dimensiones

-- La dimensión de películas

CREATE TABLE dim_film (
    film_id integer NOT NULL,
    category_name character varying(25) NOT NULL,
    actor_name character varying(90) NOT NULL,
    actor_first_name character varying(45) NOT NULL,
    actor_last_name character varying(45) NOT NULL
);

-- La dimensión de direcciones

CREATE TABLE dim_address (

```

```

        address_id integer NOT NULL,
        country character varying(50) NOT NULL,
        city character varying(50) NOT NULL
    );

-- La dimensión de alquileres

CREATE TABLE dim_rental (
    rental_id integer NOT NULL,
    date timestamp without time zone NOT NULL,
    year integer NOT NULL,
    month integer NOT NULL,
    day integer NOT NULL
);

-- La dimensión de sucursales

CREATE TABLE dim_store (
    store_id integer NOT NULL
);

--
-----
-
-- Las tabla de hechos

-- El hecho de alquileres

CREATE TABLE hechos_alquileres (
    film_id integer NOT NULL,
    address_id integer NOT NULL,
    rental_id integer NOT NULL,
    store_id integer NOT NULL,
    total_alquileres integer NOT NULL,
    monto_total numeric(5,2) NOT NULL
);

--
-----
-
-- Procedimientos para poblar las dimensiones

-- Procedimiento para poblar la tabla de hechos

CREATE OR REPLACE PROCEDURE poblar_dim_film() AS $$
DECLARE

```

```

        film_id integer;
        category_name character varying(25);
        actor_first_name character varying(45);
        actor_last_name character varying(45);
BEGIN
    FOR film_id, category_name, actor_first_name, actor_last_name IN
        SELECT film.film_id, category.name, actor.first_name,
actor.last_name
        FROM film
        INNER JOIN film_category ON film.film_id = film_category.film_id
        INNER JOIN category ON film_category.category_id =
category.category_id
        INNER JOIN film_actor ON film.film_id = film_actor.film_id
        INNER JOIN actor ON film_actor.actor_id = actor.actor_id
    LOOP
        INSERT INTO dim_film VALUES (film_id, category_name,
actor_first_name || ' ' || actor_last_name, actor_first_name,
actor_last_name);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

```

-- Procedimiento para poblar la tabla de hechos

```

CREATE OR REPLACE PROCEDURE poblar_dim_address() AS $$
DECLARE
    address_id integer;
    country character varying(50);
    city character varying(50);
BEGIN
    FOR address_id, country, city IN
        SELECT address.address_id, country.country, city.city
        FROM address
        INNER JOIN city ON address.city_id = city.city_id
        INNER JOIN country ON city.country_id = country.country_id
    LOOP
        INSERT INTO dim_address VALUES (address_id, country, city);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

```

-- Procedimiento para poblar la tabla de hechos

```

CREATE OR REPLACE PROCEDURE poblar_dim_rental() AS $$
DECLARE
    rental_id integer;

```

```

        date timestamp without time zone;
        year integer;
        month integer;
        day integer;
BEGIN
    FOR rental_id, date, year, month, day IN
        SELECT rental.rental_id, rental.rental_date, EXTRACT(YEAR FROM
rental.rental_date), EXTRACT(MONTH FROM rental.rental_date), EXTRACT(DAY
FROM rental.rental_date)
        FROM rental
    LOOP
        INSERT INTO dim_rental VALUES (rental_id, date, year, month, day);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

-- Procedimiento para poblar la tabla de hechos

CREATE OR REPLACE PROCEDURE poblar_dim_store() AS $$
DECLARE
    store_id integer;
BEGIN
    FOR store_id IN
        SELECT store.store_id
        FROM store
    LOOP
        INSERT INTO dim_store VALUES (store_id);
    END LOOP;
END;
$$ LANGUAGE plpgsql;

--
-----
-
-- Procedimientos para poblar las tablas de hechos

-- Procedimiento para poblar la tabla de hechos de los alquileres

CREATE OR REPLACE PROCEDURE poblar_hechos_alquileres() AS $$
DECLARE
    film_id integer;
    address_id integer;
    rental_id integer;
    store_id integer;
    total_alquileres integer;
    monto_total numeric(5,2);

```

```

BEGIN
    FOR film_id, address_id, rental_id, store_id, total_alquileres,
monto_total IN
        SELECT inventory.film_id, customer.address_id, rental.rental_id,
inventory.store_id, COUNT(rental.rental_id), SUM(payment.amount)
        FROM rental
        INNER JOIN inventory ON rental.inventory_id =
inventory.inventory_id
        INNER JOIN payment ON rental.rental_id = payment.rental_id
        INNER JOIN customer ON rental.customer_id = customer.customer_id
        GROUP BY inventory.film_id, customer.address_id,
rental.rental_id, inventory.store_id
        LOOP
            INSERT INTO hechos_alquileres VALUES (film_id, address_id,
rental_id, store_id, total_alquileres, monto_total);
            END LOOP;
END;
$$ LANGUAGE plpgsql;

```

```

--
-----
-
-- Truncate de tablas para el servidor 2

```

```

DO $$ DECLARE
    r RECORD;
BEGIN
    FOR r IN (SELECT tablename FROM pg_tables WHERE schemaname =
current_schema()) LOOP
        EXECUTE 'TRUNCATE TABLE ' || quote_ident(r.tablename) || '
CASCADE';
        END LOOP;
END $$;

```

```

--
-----
-
-- Pruebas

```

```

CALL poblar_dim_film();
CALL poblar_dim_address();
CALL poblar_dim_rental();
CALL poblar_dim_store();
CALL poblar_hechos_alquileres();

```

```

--

```

```
-----  
-  
-- Eliminar tablas y procedimientos  
  
DROP PROCEDURE poblar_dim_film();  
DROP PROCEDURE poblar_dim_address();  
DROP PROCEDURE poblar_dim_rental();  
DROP PROCEDURE poblar_dim_store();  
DROP PROCEDURE poblar_hechos_alquileres();  
  
DROP TABLE dim_film;  
DROP TABLE dim_address;  
DROP TABLE dim_rental;  
DROP TABLE dim_store;  
DROP TABLE hechos_alquileres;
```